

Building an HTTP Server from Scratch in C++

Complete Line-by-Line Code Explanation

This document explains every line of code in the three files that make up the HTTP server project:

[http_tcpServer_linux.h](#) | [http_tcpServer_linux.cpp](#) | [server_linux.cpp](#)

Big Picture Flow

Before diving into the code, here is the overall flow of how the server works from start to finish:

Step	What Happens
1. main()	Program starts. Creates a TcpServer object.
2. Constructor	Sets up IP, port, socket address. Calls startServer().
3. socket()	Asks the OS to create a TCP socket (a communication endpoint).
4. bind()	Attaches the socket to the given IP address and port (0.0.0.0:8080).
5. listen()	Marks socket as ready to receive incoming connections (queue of 20).
6. accept()	Waits (blocks) until a client connects, then creates a new socket for them.
7. read()	Reads the HTTP request data sent by the client into a buffer.
8. write()	Sends the HTTP response (HTML page) back to the client.
9. close()	Closes the client socket. Loops back to step 6 to wait for next client.

File 1: http_tcpServer_linux.h (The Header File)

This file **declares** the class — it describes what the class has and can do, without writing the actual logic. Think of it as a blueprint or table of contents.

Include Guard

```
#ifndef INCLUDED_HTTP_TCPSERVER_LINUX
#define INCLUDED_HTTP_TCPSERVER_LINUX
```

This is called an **include guard**. If this header file is included in multiple places, the compiler would see the class defined twice and throw an error. The guard prevents that:

- First time the file is included: `INCLUDED_HTTP_TCPSERVER_LINUX` is not yet defined, so the compiler reads the file and defines it.
- Second time: it's already defined, so everything until `#endif` is skipped.

Includes (Libraries)

```
#include <stdio.h>
#include <sys/socket.h>
#include <arpa/inet.h>
#include <stdlib.h>
#include <string>
```

- `stdio.h` — C standard input/output (printf etc.) — not strictly needed here but included for safety.
- `sys/socket.h` — provides the core socket functions: `socket()`, `bind()`, `listen()`, `accept()`.
- `arpa/inet.h` — provides IP address/port conversion tools: `inet_addr()`, `htons()`, `sockaddr_in`.
- `stdlib.h` — provides `exit()` to terminate the program.
- `string` — provides the `std::string` class for text handling.

Namespace

```
namespace http
{
```

Wraps everything inside a namespace called `http`. This means the class is referred to as `http::TcpServer`. Namespaces prevent name collisions with other code that might also define a class called `TcpServer`.

Class Declaration — Public Members

```
class TcpServer
{
public:
    TcpServer(std::string ip_address, int port);
    ~TcpServer();
    void startListen();
}
```

- `public:` — everything below this can be used from outside the class (e.g., from `main()`).
- `TcpServer(...)` — the **constructor**. Runs automatically when you create a `TcpServer` object. Takes the IP address and port to use.
- `~TcpServer()` — the **destructor**. Runs automatically when the object is destroyed. Used for cleanup (closing sockets).
- `startListen()` — the public method that kicks off the main server loop (listen → accept → read → respond).

Class Declaration — Private Members (Variables)

```
private:
    std::string m_ip_address;
    int m_port;
    int m_socket;
    int m_new_socket;
    long m_incomingMessage;
    struct sockaddr_in m_socketAddress;
    unsigned int m_socketAddress_len;
    std::string m_serverMessage;
```

`private:` — these can only be accessed from inside the class itself.

- `m_ip_address` — stores the IP address string, e.g. `"0.0.0.0"` .
- `m_port` — stores the port number, e.g. `8080` .
- `m_socket` — the main listening socket. A socket is just an integer (file descriptor) given to us by the OS.
- `m_new_socket` — a temporary socket created for each individual client connection.
- `m_incomingMessage` — declared but unused in this version of the code.
- `m_socketAddress` — a struct that holds the IP address and port in the format that socket functions require.
- `m_socketAddress_len` — the byte size of the `m_socketAddress` struct. Required by functions like `bind()` and `accept()` .
- `m_serverMessage` — the full HTTP response string that gets sent to every client.

Class Declaration — Private Methods

```
int startServer();  
void closeServer();  
void acceptConnection(int &new_socket);  
std::string buildResponse();  
void sendResponse();
```

Internal helper methods — the internal machinery of the server. Not callable from `main()`.

- `startServer()` — creates and binds the socket.
- `closeServer()` — closes sockets and exits the program.
- `acceptConnection()` — accepts one incoming client connection.
- `buildResponse()` — builds the HTTP response string once.
- `sendResponse()` — writes the response to the client socket.

```
};  
} // namespace http  
#endif
```

Closes the class, closes the namespace, and ends the include guard.

File 2: http_tcpServer_linux.cpp (The Implementation)

This file contains the **actual logic** — the full implementation of every method declared in the header.

Includes

```
#include "http_tcpServer_linux.h"
#include <iostream>
#include <sstream>
#include <unistd.h>
```

- `"http_tcpServer_linux.h"` — our own header. Uses quotes (not angle brackets) because it's a local file.
- `iostream` — provides `std::cout` for printing to the terminal.
- `sstream` — provides `std::ostringstream` for building strings from multiple parts.
- `unistd.h` — provides POSIX system calls: `read()`, `write()`, `close()`.

Anonymous Namespace — Helper Functions

```
namespace
{
    const int BUFFER_SIZE = 30720;
```

An **anonymous namespace** — everything inside is only visible within this file (like marking things as `static`). `BUFFER_SIZE = 30720` means our reading buffer is 30 KB — large enough to hold any typical HTTP request.

```
void log(const std::string &message)
{
    std::cout << message << std::endl;
}
```

A simple helper that prints any message to the terminal. `const &` means it's passed by reference (no copying) and is read-only.

```
void exitWithError(const std::string &errorMessage)
{
    log("ERROR: " + errorMessage);
    exit(1);
}
```

Logs an error message then immediately terminates the program. `exit(1)` means "exit with an error status". Zero would mean success.

Constructor

```
TcpServer::TcpServer(std::string ip_address, int port)
: m_ip_address(ip_address), m_port(port), m_socket(), m_new_socket(),
  m_incomingMessage(),
  m_socketAddress(), m_socketAddress_len(sizeof(m_socketAddress)),
  m_serverMessage(buildResponse())
```

The part after the colon `:` is the **member initializer list** — it initializes each member variable before the constructor body runs:

- `m_ip_address(ip_address)` — copies the IP string in (e.g., `"0.0.0.0"`).
- `m_port(port)` — stores the port number (e.g., `8080`).
- `m_socket()`, `m_new_socket()` — zero-initialized to safe default values.
- `m_socketAddress_len(sizeof(m_socketAddress))` — stores the byte size of the address struct.
- `m_serverMessage(buildResponse())` — calls `buildResponse()` once here and stores the result. The response is the same for every client so it's built once and reused.

```
m_socketAddress.sin_family = AF_INET;
```

Sets the address family to **IPv4**. `AF_INET` is the constant for the IPv4 Internet protocol family.

```
m_socketAddress.sin_port = htons(m_port);
```

Sets the port. `htons()` stands for **Host TO Network Short** — it converts your computer's byte order (little-endian on most modern machines) to network byte order (big-endian), which is required by all network communication standards.

```
m_socketAddress.sin_addr.s_addr = inet_addr(m_ip_address.c_str());
```

Converts the IP string `"0.0.0.0"` into a numeric format the socket understands. `c_str()` converts a `std::string` to a C-style character array, which `inet_addr()` requires.

```
if (startServer() != 0)
{
    std::ostringstream ss;
    ss << "Failed to start server with PORT: " <<
    ntohs(m_socketAddress.sin_port);
```

```
        log(ss.str());
    }
```

Calls `startServer()`. If it returns anything other than 0 (which means failure), it logs an error. `ntohs()` is the reverse of `htons()` — converts the port back to readable form for printing. `ostringstream` lets you build a string by combining text and numbers using `<<`, just like `cout`.

Destructor

```
TcpServer::~TcpServer()
{
    closeServer();
}
```

When the `TcpServer` object is destroyed (e.g., when the program ends), this runs automatically and calls `closeServer()` to free up system resources.

startServer()

```
int TcpServer::startServer()
{
    m_socket = socket(AF_INET, SOCK_STREAM, 0);
    if (m_socket < 0)
    {
        exitWithError("Cannot create socket");
        return 1;
    }
}
```

`socket()` asks the OS to create a socket and returns its file descriptor (an integer handle).

- `AF_INET` — use IPv4.
- `SOCK_STREAM` — use TCP (reliable, ordered, full-duplex byte stream).
- `0` — use the default protocol for this combination (TCP).

If it returns a negative number, socket creation failed.

```
if (bind(m_socket, (sockaddr *)&m_socketAddress, m_socketAddress_len) < 0)
{
    exitWithError("Cannot connect socket to address");
    return 1;
}
return 0;
}
```

`bind()` attaches the socket to your chosen IP address and port (like claiming a street address for your server). The cast `(sockaddr *)` is required because `bind()` expects a generic address type. Returns `0` on success.

closeServer()

```
void TcpServer::closeServer()
{
    close(m_socket);
    close(m_new_socket);
    exit(0);
}
```

- `close(m_socket)` — releases the main listening socket back to the OS.
- `close(m_new_socket)` — releases the client socket.
- `exit(0)` — ends the program successfully (0 = no error).

startListen()

```
void TcpServer::startListen()
{
    if (listen(m_socket, 20) < 0)
    {
        exitWithError("Socket listen failed");
    }
}
```

`listen()` marks the socket as passive — ready to receive incoming connections. The second argument `20` is the **backlog**: the maximum number of client connections that can wait in the queue at once. If more clients try to connect when the queue is full, they get rejected.

```
std::ostringstream ss;
ss << "\n*** Listening on ADDRESS: "
    << inet_ntoa(m_socketAddress.sin_addr)
    << " PORT: " << ntohs(m_socketAddress.sin_port) << " ***\n\n";
log(ss.str());
```

Prints the "Listening on..." banner to the terminal. `inet_ntoa()` converts the numeric IP address back to a readable string like `"0.0.0.0"`.

```
int bytesReceived;

while (true)
{
```

The **infinite server loop**. The server runs forever, handling one client per iteration.

`bytesReceived` will hold how many bytes were read from each client.

```
log("===== Waiting for a new connection =====\n\n\n");
acceptConnection(m_new_socket);
```

Logs a status message, then calls `acceptConnection()`. This call **blocks** (pauses the program) until a client actually connects. Once a client connects, `m_new_socket` is filled with a new socket dedicated to that client.

```
char buffer[BUFFER_SIZE] = {0};
```

Creates a 30 KB array of characters, all initialized to zero (`{0}`). This is where the client's HTTP request text will be stored.

```

        bytesReceived = read(m_new_socket, buffer, BUFFER_SIZE);
        if (bytesReceived < 0)
        {
            exitWithError("Failed to read bytes from client socket
connection");
        }

```

`read()` copies the client's incoming data (e.g., `GET / HTTP/1.1...`) into `buffer` and returns the number of bytes read. If the return value is negative, something went wrong.

```

        std::ostringstream ss;
        ss << "----- Received Request from client -----\n\n";
        log(ss.str());

        sendResponse();

        close(m_new_socket);
    }
}

```

Logs that a request was received, calls `sendResponse()` to write the HTML back to the client, then closes the client socket. The loop then goes back to `acceptConnection()` to wait for the next client.

acceptConnection()

```

void TcpServer::acceptConnection(int &new_socket)
{
    new_socket = accept(m_socket, (sockaddr *)&m_socketAddress,
&m_socketAddress_len);
}

```

`accept()` takes the next waiting client from the queue and returns a **brand-new socket** dedicated to communicating with that specific client. `int &new_socket` is passed by reference — assigning to it here changes `m_new_socket` in the caller.

```

    if (new_socket < 0)
    {
        std::ostringstream ss;
        ss << "Server failed to accept incoming connection from ADDRESS: "
            << inet_ntoa(m_socketAddress.sin_addr) << "; PORT: "
            << ntohs(m_socketAddress.sin_port);
        exitWithError(ss.str());
    }
}

```

If accepting failed (negative return), builds a descriptive error message including the client's address and port, then exits.

buildResponse()

```
std::string TcpServer::buildResponse()
{
    std::string htmlFile = "<!DOCTYPE html><html lang=\"en\"><body><h1> HOME </h1><p> Hello from your Server :) </p></body></html>";
```

The HTML page content that the server will serve to every client. The backslash-quote `\"` is an escaped quote character inside a C++ string.

```
std::ostringstream ss;
ss << "HTTP/1.1 200 OK\nContent-Type: text/html\nContent-Length: "
    << htmlFile.size() << "\n\n"
    << htmlFile;
return ss.str();
}
```

Builds a valid HTTP response. HTTP responses have a specific format:

- `HTTP/1.1 200 OK` — status line. `200` means "everything is fine".
- `Content-Type: text/html` — tells the browser the body is HTML.
- `Content-Length: ...` — tells the browser how many bytes to expect.
- `\n\n` — a blank line that **separates headers from the body** (required by the HTTP standard).
- Then the HTML content follows.

sendResponse()

```
void TcpServer::sendResponse()
{
    long bytesSent;
    bytesSent = write(m_new_socket, m_serverMessage.c_str(),
m_serverMessage.size());
```

`write()` sends the response bytes over the client's socket. It takes the socket, the data as a C-string, and the number of bytes to send. It returns how many bytes were actually sent.

```
if (bytesSent == m_serverMessage.size())
{
    log("----- Server Response sent to client -----\n\n");
}
else
{

```

```
        log("Error sending response to client");  
    }  
}
```

Compares bytes sent vs bytes in the message. If they match, the full response was delivered successfully. If not, something went wrong mid-transfer.

File 3: server_linux.cpp (The Entry Point)

This is the simplest file — it just starts the server.

```
#include "http_tcpServer_linux.h"
```

Includes the header so `main()` knows about the `TcpServer` class.

```
int main()
{
    using namespace http;
```

`main()` is where every C++ program starts. `using namespace http` lets us write just `TcpServer` instead of `http::TcpServer` for cleaner code.

```
TcpServer server = TcpServer("0.0.0.0", 8080);
```

Creates a `TcpServer` object. This triggers the constructor, which:

- Sets up the IP and port.
- Configures the socket address struct.
- Calls `startServer()` which creates and binds the socket.
- Builds the HTTP response string once.

`"0.0.0.0"` means the server will accept connections on *any* network interface of the machine.

```
server.startListen();
```

Starts the infinite loop: listens for connections, accepts them one at a time, reads requests, and sends responses. This call never returns (the loop runs forever).

```
    return 0;
}
```

This line is never actually reached because the loop above runs forever. It's there to satisfy the C++ requirement that `main()` returns an integer.

Summary — Key Concepts

Concept	Meaning
Socket	An integer handle the OS gives us to represent a network communication endpoint.
bind()	Claims an IP address and port for the server — like registering a street address.
listen()	Tells the OS to start queuing incoming connection attempts.
accept()	Takes one connection from the queue and returns a new socket for that client.
read()	Reads the client's HTTP request from the socket into a buffer.
write()	Sends the HTTP response back to the client through the socket.
htons()	Converts port number from host byte order to network byte order.
inet_addr()	Converts an IP string like "0.0.0.0" to a numeric format sockets understand.
namespace	Groups related code to avoid naming conflicts with other code.
Include guard	Prevents a header file from being included more than once in a compilation.
Constructor	Automatically runs when an object is created — sets everything up.
Destructor	Automatically runs when an object is destroyed — cleans everything up.
Reference (&)	Passes a variable by reference so the function can modify the original.
BUFFER_SIZE	30 KB buffer used to hold the incoming HTTP request text from each client.

Rule of thumb for includes:

Use angle brackets `< >` for system/standard library headers (e.g. `<iostream>`).

Use quotes `" "` for your own local header files (e.g. `"http_tcpServer_linux.h"`).